¿Cómo afrontar la práctica de distribuidos?

Sin pegarse un tiro ni pegarselo a nadie. Documento en redacción, revisa última versión.

Gonzalo

Marzo 2026

DOCUMENTACIÓN

- Estas diapositivas:
  https://ggelado.github.io/diaposDistribuidos/diapos.pdf
- En documento para leer:
  https://ggelado.github.io/diaposDistribuidos/documento.docx

Este documento está en redacción, cada cambio actualiza los ficheros en esos enlaces.

Consulta regularmente la última versión en esos enlaces.

INTRODUCCIÓN

¿DE QUÉ VA ESTO?

Va de que los 4 nodos de triqui (p.ej.) hablen entre ellos, se manden ficheritos, mensajitos y demás, pero que ninguno sea un servidor, **TODOS SON SERVIDORES Y CLIENTES**.

Eso es el P2P (lo que se hacía hace años con los torrents ilegales).

Ya lo iremos definiendo sobre la marcha.

¿QUÉ NECESITO?

- Entorno linux:
  - WSL
  - Máquina Virtual
  - Linux nativo
  - ¿Triqui? ¿Escritorios virtuales? (si tu paciencia y cordura lo permiten)
- Conexión con triqui
- Usar el Gestor de Entregas de Triqui (ya explicaré cómo funciona)

¿EMPEZAMOS?

1. Coge un editor cómodo (vas a dedicarle un ratito)
   - VSCode
   - CLion
   - ¿Nano? ¿vi? ¿emacs?

2- Pon tu entorno de trabajo

3- Enciende la VPN

ARRANCANDO

ARRANCANDO

Recuerda: Ya sabes que copiar texto de un PDF a una terminal puede hacer cosas raras.
Saltos de línea que no debes poner, caracteres raros, comillas… Ten cuidado al copiar y revisa bien. En algunas diapositivas se divide el texto en 2 líneas para que no se corte, eso no significa que en tu terminal deban ser 2 líneas.

FICHEROS DE TRABAJO

Ejecuta en una terminal

```
wget https://laurel.datsi.fi.upm.es/_media/docencia/asignaturas
/sd/ring-2026.tgz
# 2 líneas para que no se corte
tar xvfz ring-2026.tgz
cd DATSI/SSDD/ring.2026/src/
```

De una

COMUNICACIÓN CON TRIQUI

Vas a pasar muchas veces ficheros de triqui a tu ordenador y viceversa, puedes hacer SCP todo el rato, pero vas a deprimirte.

- Bitvise (Mi preferido para Windows)
- Uso Ubuntu nativo

O si no…

```
scp ficheroOrigen usuario@triqui.fi.upm.es:
~/DATSI/SSDD/ring.2026/src/

# 2 líneas para que no se corte
```

Usa esos programas, por favor, te salvan la vida.
(Los nombres son enlaces, puedes hacer click en ellos)

GESTOR DE ENTREGAS

Funciona igual que en SSOO:

ADVERTENCIA: Por alguna razón el Gestor espera que los ficheros estén en `~/DATSI/SSDD/ring.2026/` pero los profes han puesto el código de la plantilla en `todoEso/src`, por lo que **falla**. Saca los ficheros de src y llévalos a esa carpeta.

Con Bitvise y demás es sencilla la operación.

GESTOR DE ENTREGAS (2)

Cada vez que queramos hacer una entrega:

1. Entrar en triqui
2. Poner **DONDE SEA** (en una terminal): `entrega.sd ring.2026`
3. Introducir el número de matrícula cuando se solicite
4. El corrector corre cada 3h y manda un correo

A PICAR CÓDIGO

¿DÓNDE SE TOCA?

- ring_cln.c
- ring_srv.c
- common.c
- include/common.h

Y ya está, el resto de ficheros se miran pero no se tocan.

¿CÓMO EMPIEZO?

Pues por el principio.

No empieces a leerte todo el enunciado, vamos por partes:

ESQUEMA BÁSICO DE CONEXIÓN (ESTO HAY QUE TATUARSELO)

```
/* CLIENTE */

// Conectamos con server
int s = create_socket_cln(remote_ip, remote_port);

// Si hay error retornar algo que no sea cero
if (s < 0) return ERROR;

char op_code = OPERACION; // char, int...
if (send(s, &op_code, sizeof(char), 0) != sizeof(char)) {
    perror("mensaje_error");
    close(s);
    return ERROR;
}
// Después a enviar y recibir datos
// y demás con send y recv
```

ESQUEMA BÁSICO DE SERVIDOR (LO MISMO, GRABARSELO)

```
switch (cod_operacion) {
        case EL_QUE_SEA: {
            // Tramito operacion
            // Con recv recibo datos adicionales
            // Con send envío respuestas
            break;
```

FASE 1 - PASO 1

INTERFACES

```
// inicia el nodo añadiéndolo a la red P2P si ya está creada;
// los puertos e IPs deben estar en formato de red;
// debe devolver en el último parámetro el puerto reservado
// ¡¡¡OJO A ESTO ÚLTIMO!!!: en formato red;
// retorna 0 si OK y -1 si error
int ring_init(const char *shared_dir, unsigned int local_ip,
    unsigned int remote_ip, unsigned short remote_port,
    unsigned short *alloc_port);

// función local que devuelve la IP y el puerto del nodo
int ring_self(unsigned int *ip, unsigned short *port);
```

DESPLEGAR PARTE SERVIDORA

Vamos con el fichero `ring_cln.c`.

Objetivo:

- Debe guardar la información recibida en sus parámetros e implementarse la función ring_self a partir de ella.
- Debe crear el socket de servicio, para lo que puede usar la función create_socket_srv, tal como se hace en el ejemplo propuesto. Esa función devuelve el puerto seleccionado por el sistema operativo en formato de red, que, a su vez, debe ser devuelto por esta función.
- Debe crear el *thread* de servicio que ejecutará la función server_thread de ring_srv.c pasándole como argumento el socket de servicio. Para ello, puede usar la función create_thread de common.c, que crea un *thread* de tipo *detached*.

¿QUÉ TENEMOS?

Nos dan esto

```
// inicia el nodo añadiéndolo a la red P2P si ya está creada;
// los puertos e IPs deben estar en formato de red;
// debe devolver en el último parámetro el puerto reservado en fo
// retorna 0 si OK y -1 si error
int ring_init(const char *shrd_dir, unsigned int local_ip, unsign
    if (initialize()) return -1; // ya está inicializada
    return 0;
}
```

Hay params de **salida** (punteros):

- `unsigned short *alloc_port` es de **salida**

DEBE GUARDAR LA INFORMACIÓN RECIBIDA EN SUS PARÁMETROS

Pues vamos a hacer eso, eso es sencillo.

Variable arriba y listo:

```
shared_dir_copia = strdup(shrd_dir);
ip_copia = local_ip;
```

`strdup` para copiar los *Strings*.

E IMPLEMENTARSE LA FUNCIÓN `ring_self` A PARTIR DE ELLA.

`ring_self` devuelve los datos propios.

```
int ring_self(unsigned int *ip, unsigned short *port)
```

Son parámetros de salida (punteros), ya sabes cómo va

```
if (!is_initialized()) return -1; // no está inicializada
*ip = ip_copia;
// *port = port_copia; // ni caso, ya lo veremos
return 0;
```

Recuerda: Self. Retornamos la PROPIA, no la remota.

PRIMER PÁRRAFO SUPERADO

DEBE CREAR EL SOCKET DE SERVICIO, PARA LO QUE PUEDE USAR LA FUNCIÓN **CREATE_SOCKET_SRV**, TAL COMO SE HACE EN EL EJEMPLO PROPUESTO.

¿Qué ejemplo?

Pues este

```
int main(int argc, char *argv[]) {
    int s, s_conec;
    unsigned int addr_size;
    unsigned short port;
    struct sockaddr_in clnt_addr;

    // inicializa el socket y lo prepara para aceptar conexiones
    if ((s=create_socket_srv(&port)) < 0) return 1;
    printf("Reservado el puerto %d\n", ntohs(port));
```

Ejemplo: `server.c`

VAMOS CON ELLO, A COPIAR SIN PIEDAD

```
if ((s=create_socket_srv(&port_copia)) < 0) return 1;
```

(por cierto, ya tenemos el valor de retorno de `*alloc_port`)

Ya podemos completar en ring_self

(además, recuerda que tienes que guardar ese puerto en alguna variable)

DEBE CREAR EL *thread* DE SERVICIO QUE EJECUTARÁ LA FUNCIÓN SERVER_THREAD DE RING_SRV.C PASÁNDOLE COMO ARGUMENTO EL SOCKET DE SERVICIO. PARA ELLO, PUEDE USAR LA FUNCIÓN CREATE_THREAD DE COMMON.C, QUE CREA UN *thread* DE TIPO *detached*.

¿Cómo? ¿Qué pone ahí?

Poco a poco, no nos asustemos:

Vamos a buscar el ejemplo ese que dice, a ver si vemos algo.

Por cierto, ahí te lo dice pero… es en `ring_srv.c`.

A VER EL EJEMPLITO

```
while(1) {
    addr_size=sizeof(clnt_addr);
    // acepta la conexión
    if ((s_conec=accept(s, (struct sockaddr *)&clnt_addr, &addr_s
        perror("error en accept"); close(s); return 0;
    }
    printf("conectado cliente con ip %s y puerto %u\n",
            inet_ntoa(clnt_addr.sin_addr), ntohs(clnt_addr.sin_po
    // crea el thread de servicio pasándole el argumento por valo
    create_thread(request_handler, (void *)(long)s_conec);
}
close(s); // cierra el socket general
```

Pues ya sabes, A COPIAR SIN PIEDAD

PD: Lo de formato de red se encargan `inet_ntoa` y `ntohs` (esto es de SSOO, pero no hace falta, tú copia y no preguntes).

DEBE CREAR EL *thread* DE SERVICIO

```
create_thread(
```

QUE EJECUTARÁ LA FUNCIÓN SERVER_THREAD DE RING_SRV.C

```
create_thread(request_handler
```

PASÁNDOLE COMO ARGUMENTO EL SOCKET DE SERVICIO

```
create_thread(request_handler, (void *)(long)s_conec);
```

Si es que es calcado, literalmente, **COPIA LO QUE VIENE EN EL EJEMPLO**.

NOS VAMOS AL CÓDIGO

```
// función para el thread que implementa la funcionalidad de serv
// debe recibir como argumento el socket de servicio
void *server_thread(void *arg){
    return NULL;
}
```

y a hacer lo mismo.

Acuerdate de dar de alta variables y demás para que compile, y que en vez de retornar 0 retornamos NULL (es `void`), PERO EL RESTO IGUAL.

PRUEBA DEL PASO 1

```
mkdir dir1 && ./ring dir1
```

En otra terminal:

```
ss -tap | grep <puerto>
# debe aparecer LISTEN con el PID de ring
```

FASE 1 - PASO 2

PRIMERA OPERACIÓN REMOTA

Objetivo: que un nodo le pregunte el PID a otro.

Sin parámetros de entrada, devuelve un `int`.

CÓDIGOS DE OPERACIÓN

Antes de enviar nada hay que decirle al servidor **qué quieres**. Opciones:

- Carácter: `'P'`
- Entero: `0`
- String: `"GETPID"`

El `char` y el entero son los más cómodos para `switch` (en C no podemos hacerlo con un string).

PARTE CLIENTE: `ring_remote_pid`

```
// socket ya creado
char op = 'P';
send(s, &op, sizeof(char), 0); # ¿algún flag?

int pid;
recv(s, &pid, sizeof(int), MSG_WAITALL); // ¿WAITALL?
close(s);
return ntohl(pid);
```

PARTE SERVIDORA

```
char op;
recv(soc, &op, sizeof(char), MSG_WAITALL);

switch(op) {
    case 'P': {
        int pid = htonl(getpid());
        write(soc, &pid, sizeof(int)); // o send casi que mejor
        break;
    }
}
close(soc);
return NULL;
```

PRUEBA DEL PASO 2

```
./ring dir1
# pulsa P, introduce la IP y puerto del propio nodo

PID 2576024
```

Debe coincidir con el PID del mensaje de bienvenida.

FASE 2

FASE 2

Ahora ya hay que ir soltándose un poco, ya no puedo darte todo el código.

DEBE GESTIONARSE LA INFORMACIÓN DE QUÉ NODO ES EL SUCESOR

- ¿Quién es el sucesor?: Si soy el primero, pues yo mismo.
- ¿Y si no? Pues se lo pregunto al nodo al que me conecto.
  - Nueva operación en el servidor.
  - Le envío los datos del sucesor actual que tenga guardado este nodo.

E IMPLEMENTARSE LA FUNCIÓN RING_SUCCESSOR A PARTIR DE ELLA.

```
// función local que devuelve la IP y el puerto del nodo sucesor;
// retorna 0 si OK y -1 si error
int ring_successor(unsigned int *ip, unsigned short *port) {
```

Es un `getter`, si has hecho el anterior, este es **TRIVIAL** (como odio esa palabra).

SIGAMOS LEYENDO EL ENUNCIADO

- Hay que definir una nueva operación que permita dar de alta un nuevo nodo en una red ya existente.
- Esa operación de alta debe enviar al nodo de contacto la información del nuevo nodo, que pasará a ser el nuevo sucesor de dicho nodo de contacto, y responder al nuevo nodo con la información del sucesor actual del nodo de contacto, que pasará a ser el sucesor del nuevo nodo. Nótese que es suficiente con que el nuevo nodo envíe su puerto de servicio, no necesitando mandar su dirección IP, ya que esta le llega al nodo de contacto como parte del protocolo TCP (la propia función accept o, posteriormente, getpeername devuelven la dirección IP del nodo que se conecta).

- En la función inicial (ring_init) de la parte cliente (ring_cln.c), en caso de querer incorporarse a una red ya existente, debe solicitarse esa nueva operación, conectándose al nodo de contacto, enviándole el código de operación y el puerto de servicio, quedándose a la espera de la respuesta que permitirá actualizar el sucesor de ese nuevo nodo, y, finalmente, cerrando el socket.
- En la parte servidora (ring_srv.c), hay que ajustar su sucesor con los valores correspondientes al nuevo nodo y responder con los que tenía previamente como sucesor.

Pero si ya lo hemos hecho. Venga, vamos a seguir.

OPERACIÓN SUCESOR DE UN NODO REMOTO

El objetivo de esta segunda operación remota es incorporar una operación que devuelva la dirección IP y puerto de un nodo, de manera que, más adelante, nos sirva de base para la operación *lookup*, que devuelve esa misma información.

Similar a la primera operación remota, que obtenía el PID del proceso remoto especificado, pero, en este caso, recibe la información (IP y puerto) del sucesor de dicho nodo, en vez de un entero con el PID. Ese código hay que incorporarlo en la función get_remote_successor y en la parte servidora.

Traducción: Cuando nos pidan el sucesor devolver el actual. Más o menos lo que hemos hecho en la operación de servidor con `ring_init`. Desde el servidor recibimos la petición y le mandamos el sucesor.

IMPLEMENTACIÓN DE LA PRIMERA OPERACIÓN QUE USA EL ANILLO

- Un usuario en un determinado nodo N1 solicita la operación que obtiene el sucesor del sucesor de un determinado nodo que especifica (N2), invocando la función ring_remote_successor_successor.
- Como parte del código de esa función, el lado cliente de N1 envía a la parte servidora de N2 el mensaje con esa petición y se queda esperando la respuesta. Nótese que esta parte tiene el mismo comportamiento que las dos operaciones remotas introducidas hasta ahora.

- La parte servidora de N2, al detectar esta operación, debe ejercer el rol de cliente (este es el punto donde cambia el patrón) enviando la operación sucesor (y no sucesor de sucesor), que se implementó en el paso previo, a su nodo sucesor (N3), y quedándose a la espera de la respuesta. Cuando reciba la respuesta, se la enviará a N1, que estaba esperándola.
- La parte servidora de N3 responde a la petición de sucesor.

¿CÓMO SE HACE?

- **Hay que implementar una nueva operación remota.**
- En la función ring_remote_successor_successor de la parte cliente, hay que incorporar la nueva operación. Dado que tiene las mismas características que la operación anterior (no tiene parámetros y devuelve IP y puerto), es igual cambiando solamente el código de operación.

- La parte servidora, sin embargo, cambia respecto a las operaciones anteriores ya que tiene que ejercer de cliente enviando una operación de sucesor al siguiente nodo del anillo, esperando su respuesta y enviando esta al nodo cliente original. Nótese que, dado que la parte servidora está ejerciendo de cliente, es posible usar en esta la función ring_remote_successor del lado cliente para implementar parte de esta funcionalidad.

Traducción: Le pregunto a mi sucesor cuál es su sucesor con la op. del aptdo. anterior, y cuando me conteste le contesto al cliente con lo que me diga.

Y YA ESTÁ

FASE 3

FASE 3

Ya has hecho las anteriores fases, ahora ya no te puedo guiar tanto, pero Costoya, que es un grande, ya lo ha hecho:

`copy_read_mmap.c`

Literalmente ahí viene hecho para el cliente, y `copy_sendfile.c` para el servidor.

FASE 4

FASE 4

Esto es como si fuera recursividad: Le pregunto a mi sucesor si tiene el fichero. Si lo tiene me responde. Si no lo tiene, le pregunta a su sucesor (si no se agota el TTL llamado `hops`).
Cuando llegue al caso base, que tenga el fich, responde y se resuelven todas las peticiones pendientes.

FASE 5

FASE 5

NO TIENES QUE HACERLA

Si te funciona lo de antes, **ENHORABUENA, HAS TERMINADO**.

A CURRAR

HAZ FUNCIONAR ESTO

bash in ggelado

```
ggelado  ~  ssh gonzalo.gelado@triqui1.fi.upm.es
gonzalo.gelado@triqui1.fi.upm.es's password:
 _        _             _ _    __ _
| |_ _ __(_) __ _ _   _(_) |  / _(_) _   _ _ __  _ __ ___    ___  ___
| __| '__| |/ _` | | | | | | | |_| || | | | '_ \| '_ ` _ \  / _ \/ __|
| |_| |  | | (_| | |_| | | |_|  _| || |_| | |_) | | | | | | ||  __/\__ \
 \__|_|  |_|\__, |\__,_|_|_(_)_| |_(_)__,_| .__/|_| |_| |_(_)___||___/
               |_|                        |_|

Welcome to Debian GNU/Linux 12 (bookworm) (6.1.0-43-amd64).

System information as of: Wed Mar  4 19:36:23 CET 2026

System Load:    0.58    IP Address:     10.2.0.146
Memory Usage:   14.4%   System Uptime:  9 days
Usage On /:     11%     Swap Usage:     0.1%
Local Users:    5       Processes:      749
```

pwsh in distribuidos / bash in ggelado

```
System Load:    0.06    IP Address:     10.2.0.147
Memory Usage:   4.7%    System Uptime:  9 days
Usage On /:     13%     Swap Usage:     0.0%
Local Users:    1       Processes:      511

Linux triqui2.fi.upm.es 6.1.0-43-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6
.1.162-1 (2026-02-08) x86_64

The programs included with the Debian GNU/Linux system are free softwa
re;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Mar  4 19:35:15 2026 from 138.100.242.96
gonzalo.gelado@triqui2:~$
```

bash in ggelado

```
System Load:    0.38    IP Address:     169.254.210.255
Memory Usage:   5.4%    System Uptime:  9 days
Usage On /:     17%     Swap Usage:     0.0%
Local Users:    0       Processes:      531

Linux triqui4.fi.upm.es 6.1.0-43-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6
.1.162-1 (2026-02-08) x86_64

The programs included with the Debian GNU/Linux system are free softwa
re;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Mar  4 15:19:04 2026 from 138.100.242.98
gonzalo.gelado@triqui4:~$
```

bash in ggelado

```
System Load:    0.04    IP Address:     138.100.240.41
Memory Usage:   7.6%    System Uptime:  9 days
Usage On /:     12%     Swap Usage:     0.0%
Local Users:    2       Processes:      627

Linux triqui3.fi.upm.es 6.1.0-43-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6
.1.162-1 (2026-02-08) x86_64

The programs included with the Debian GNU/Linux system are free softwa
re;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Mar  4 19:35:26 2026 from 138.100.242.96
gonzalo.gelado@triqui3:~$
```

EXPERIENCIA PERSONAL

- Fase 1: Difícil (no sabes cómo empezar).
- Fase 2: Es seguir lo que estabas haciendo, sencilla.
- Fase 3: Tiene algo de lío porque tienes que proyectar ficheros y demás (`mmap` y todo eso).
- Fase 4: Bastante sencilla la verdad. La lógica es ir preguntando al sucesor restando 1 en `hops`.

AVANZADO

AVANZADO

Esto es algo adicional, no es puramente de la práctica, y puedes omitirlo directamente, pero si utilizas git, y tienes algo de destreza, puede llegar a ser muy cómodo:

ENTREGANDO CON GIT PUSH

¿Sabías que triqui puede ser un servidor git?

Sí, no solo GitHub y GitLab son servidores git. Puedes crear un servidor git del siguiente modo:

1. Ubica en triqui el directorio `~/DATSI/SSDD/ring.2026`
2. Da de alta un repositorio (ya sabes, `git init`. Recuerda que las ramas deberían llamarse igual para evitar problemas más adelante, aunque no es fundamental.)
3. Regístralo como `origin` en tu carpeta de trabajo.

AGREGANDO ORÍGENES A GIT

Tienes 2 opciones: Solo usar triqui como origen, o usar tanto triqui como github.

- `git remote add [nombre] [url]`

Donde `url` es `ssh://usuario@triqui1.fi.upm.es/homefi/alumnos/rutaHOME/DATSI/SSDD/ring.2026`
Recuerda consultar la ruta completa con el mandato

pwd

Puedes llamar al remote origin, o simplemente algo como triqui (tendrás que hacer `git push triqui`)

PUSH COMBINADO

```
git remote add origin https://repo1.git # p.ej. github
git remote set-url --add --push origin # url triqui de antes
```

Nota: Es posible que tengas que configurar el upstream

```
git push --set-upstream origin main # o master
```

ENTREGA AUTOMATIZADA

La gracia de todo esto es que puedas entregar de forma automática al hacer push. Para ello:

1. Accede a `~/DATSI/SSDD/ring.2026/.git/hooks`
2. Crea un fichero `post-receive`

Ese fichero será un script bash con instrucciones a ejecutar cada vez que reciba un push. En nuestro caso, queremos que corra el script de entrega:

POST-RECEIVE HOOK

```bash
#!/bin/bash
set -euo pipefail
cd "/homefi/alumnos/TUHOME/DATSI/SSDD/ring.2026"
```

```bash
if ! printf "[redacted]\ns\ns\n" | script -qc "stty -echo; entrega.sd ring.2026" /dev/null 2>&1; then
  echo "Falló la ejecución de entrega.sd. Commit rechazado."
  exit 1
fi
```

La parte roja sustitúyela por tu número de matrícula (`numMat\ns\ns\n`)

Eso lo que hace es seguir los pasos del corrector:

1. Número de matrícula.
2. Confirmar entrega.
3. Sobrescribir entrega.